RHLC Community Exporter — User Guide

Export your Red Hat Learning Community posts from `learn.redhat.com` to a self-contained offline HTML archive with all images and attachments downloaded locally.

Prerequisites

- **uv** (recommended) — install from https://docs.astral.sh/uv/getting-started/installation/
- **Python 3.11+** — uv can install this for you (see below)
- **git** — to clone the repository
- A `learn.redhat.com` account (Red Hat SSO)
- `my_community_content.json` — your community content export *(not needed if you use `--auto`, which downloads it for you)*

1. Clone the repository

```
git clone https://github.com/tmichett/rhlc_save.git
cd rhlc_save
```

2. Install Python 3.11+ with uv (if needed)

If you don't have Python 3.11 or later installed, uv can install it for you:

```
# Install Python 3.11 using uv
uv python install 3.11

# Verify installation
uv python list
```

Alternatively, you can install Python manually:

- **RHEL/Fedora:** `sudo dnf install python3.11`
- **Ubuntu/Debian:** `sudo apt install python3.11`
- **macOS:** `brew install python@3.11`
- **Windows:** Download from https://www.python.org/downloads/

3. Install dependencies (first run only)

With uv (recommended):

```
uv sync
```

Or with plain pip:

```
pip install -r requirements.txt
```

For the fully-automatic mode (`--auto` / `--fetch-json`), also install Playwright:

```
uv pip install playwright
uv run playwright install chromium
```

Quick Start

Option A — Fully automatic (recommended)

The easiest path: one command logs you in, downloads your community JSON, and runs the full export pipeline automatically.

Requires Playwright (install once):

```
uv pip install playwright
uv run playwright install chromium
```

Then run:

```
uv run export_community.py --auto
```

What `--auto` does:

1. Opens a real Chromium browser window at `learn.redhat.com`
2. **Click "Sign In"** in the top-right corner and log in with your Red Hat account
3. Once fully logged in, press Enter in the terminal
4. The script navigates to your **Advanced Profile** page automatically
5. Clicks **My community content** to download the JSON
6. Saves your session cookies to `cookies.txt` for future runs
7. Runs the full export pipeline (images, attachments, HTML generation)

> ⚠️ **Important:** When the browser opens, you will see the RHLC homepage — **not** a login form. You must click **Sign In** in the top-right corner yourself. The `/t5/s/sso` path is not a valid direct login URL on this site.

> **Tip:** On subsequent runs, if `my_community_content.json` already exists and your cookies are still valid, you can skip the download step:
>
> ```
> uv run export_community.py --cookies cookies.txt
> ```

To download a fresh JSON without re-running the full export:

```
uv run export_community.py --fetch-json
```

Option B — Cookie file (manual, most reliable for repeated runs)

`learn.redhat.com` is a JavaScript single-page application. Its login flow cannot be automated with a plain HTTP client. If you already have `my_community_content.json`, the most reliable method is to export your browser session cookies after logging in manually.

1. Log in to https://learn.redhat.com in your browser
2. Install the **Cookie-Editor** browser extension (Chrome / Firefox)
3. Click the extension icon → **Export** → **Netscape** format
4. Save the file as `cookies.txt` in this directory
5. Run:

```
uv run export_community.py --cookies cookies.txt
```

To save the cookies for future runs automatically, add `--save-cookies`:

```
uv run export_community.py --cookies cookies.txt --save-cookies
```

Option C — Playwright browser login (interactive, no pre-existing JSON)

If you have Playwright installed but want to log in interactively without using `--auto`, run without `--cookies`:

```
uv run export_community.py --save-cookies
```

A Chromium browser window will open at `learn.redhat.com`.

> ⚠️ **Important:** Click **Sign In** in the top-right corner of the page and log in with your Red Hat account. Do not navigate away. Once you are fully logged in, come back to the terminal and press Enter.

The script captures your session cookies and continues downloading. Use `--save-cookies` to write them to `cookies.txt` for future runs.

> **Note:** This option does **not** download `my_community_content.json` automatically. Use `--auto` or `--fetch-json` for that.

Option D — No authentication (HTML only, images may be broken)

```
uv run export_community.py --no-auth
```

Post HTML files are generated from the JSON. Images that require authentication will appear as `[Image: <id>]` placeholders.

Output Structure

After running, the `output/` directory will contain:

```
output/
├── rhtlc_main.html          <- Master index — open this in your browser
├── assets/
│   ├── highlight.min.js     <- Syntax highlighting (downloaded once)
│   └── highlight.min.css
├── images/
│   └── <image-id>.png/jpg/  <- All post images
├── attachments/
│   └── <filename>           <- All post attachments
├── posts/
│   └── <Forum-Subject>.html <- One file per post
└── download_errors.log      <- Only created if any downloads failed
```

Open `output/rhtlc_main.html` in any browser to browse your archive. All links are relative — the entire `output/` folder is portable.

CLI Reference

```
usage: export_community.py [-h] [--auto] [--fetch-json]
                           [--json FILE] [--output DIR]
                           [--cookies FILE] [--save-cookies]
                           [--skip-images] [--skip-attachments]
                           [--skip-assets] [--no-auth]
```

Flag	Default	Description
`--auto`	off	**Fully automatic**: log in via browser, download JSON, save cookies, run full export. Requires Playwright.
`--fetch-json`	off	Log in via browser and download `my_community_content.json` only, then run the export. Requires Playwright.
`--json FILE`	`my_community_content.json`	Path to the JSON dump
`--output DIR`	`output`	Directory to write all output files
`--cookies FILE`	*(none)*	Netscape cookies.txt file for authentication
`--save-cookies`	off	Save session cookies to `cookies.txt` after Playwright login (implied by `--auto`)
`--skip-images`	off	Skip downloading images (use existing files in `output/images/`)
`--skip-attachments`	off	Skip downloading attachments
`--skip-assets`	off	Skip downloading highlight.js assets
`--no-auth`	off	Skip all authentication (images behind login will not download)

Authentication Details

Why Authentication Is Needed

Images embedded in Khoros posts are served from URLs like:

```
https://learn.redhat.com/t5/image/serverpage/image-id/12488i.../image-size/original
```

These URLs require an active session cookie. Without authentication, the server returns a login redirect instead of the image.

Why Form-Based Login Does Not Work

`learn.redhat.com` is a JavaScript single-page application (SPA). The login page is rendered entirely by JavaScript — there is no HTML form accessible to a plain HTTP client like `requests`. Attempting to POST credentials directly results in a 404 or empty response.

Cookie File Method (Recommended)

The `--cookies` flag accepts **Netscape/Mozilla** format cookie files — the standard format exported by browser extensions like Cookie-Editor.

The file starts with:

```
# Netscape HTTP Cookie File
```

Steps:

1. Log in to https://learn.redhat.com in Chrome or Firefox
2. Open Cookie-Editor extension → Export → Netscape format
3. Save as `cookies.txt`
4. Run: `uv run export_community.py --cookies cookies.txt`

Cookies typically expire after a few hours to days. If images stop downloading, export fresh cookies and re-run.

Playwright Browser Method

When Playwright is installed and no `--cookies` file is provided, the script opens a real Chromium browser window at `learn.redhat.com`.

> ⚠️ **Important:** The browser opens at the **homepage** — not a login form. You must click **Sign In** in the top-right corner yourself and complete the Red Hat SSO login. Once you are fully logged in, return to the terminal and press Enter.

The script extracts all cookies from the browser context and uses them for image and attachment downloads.

Install Playwright once:

```
uv pip install playwright && uv run playwright install chromium
```

Re-running / Resuming

The script is **idempotent** — it skips files that already exist:

- Images already in `output/images/` are not re-downloaded
- Attachments already in `output/attachments/` are not re-downloaded
- `highlight.js` assets are not re-downloaded if present

To regenerate only the HTML (after editing the script's templates):

```
uv run export_community.py --skip-images --skip-attachments --skip-
assets --no-auth
```

To retry failed image downloads (re-run normally — existing files are skipped):

```
uv run export_community.py --cookies cookies.txt
```

Troubleshooting

RHEL 10 / No Display Server Available

If you see an error like:

```
Target page, context or browser has been closed
Browser logs:
╔═══════════════════════════════════════════════════════════════════════════
═══════════════════════╗
║ Looks like you launched a headed browser without having a XServer
running.                     ║
║ Set either 'headless: true' or use 'xvfb-run <your-playwright-app>'
before running Playwright.║
╚═══════════════════════════════════════════════════════════════════════════
═══════════════════════╝
```

This means Playwright cannot launch a GUI browser because no display server is available. This commonly occurs on:

- RHEL 10 systems without X11 configured
- Headless servers
- SSH sessions without X forwarding
- Systems running Wayland without XWayland

Solutions (in order of simplicity):

Option 1: Set DISPLAY for GNOME/Wayland (Easiest for RHEL 10 Desktop)

RHEL 10 uses GNOME with Wayland by default. XWayland provides X11 compatibility, so you just need to set the DISPLAY variable:

```
export DISPLAY=:0
uv run export_community.py --auto
```

This tells Playwright to use your existing desktop session. The browser window will appear on your screen.

Option 2: Use Xvfb (Virtual Display)

Install and use a virtual X server that runs in the background:

```
# Install Xvfb (one-time setup)
sudo dnf install xorg-x11-server-Xvfb

# Run with virtual display
xvfb-run uv run export_community.py --auto
```

The browser runs invisibly in the background. You'll still need to interact with it (the script pauses for you to log in), but you won't see the window.

Option 3: Cookie Export Method (Works Anywhere)

This method works on any system, including headless servers:

1. On a machine with a GUI (your laptop, another workstation, or via VNC):
   - Open Firefox/Chrome and go to https://learn.redhat.com
   - Log in with your Red Hat account
   - Install the **Cookie-Editor** browser extension
   - Click the extension → **Export** → **Netscape** format
   - Save as `cookies.txt`
2. Transfer `cookies.txt` to your RHEL 10 machine:

   ```
   scp cookies.txt user@rhel10-machine:~/rhlc_save/
   ```

3. Run the exporter with cookies:

   ```
   uv run export_community.py --cookies cookies.txt
   ```

This method is also the most reliable for repeated runs since cookies can be reused until they expire.

Option 4: SSH X Forwarding

If connecting from another machine with X11:

```
# From your local machine with X server
ssh -X user@rhel10-machine
cd rhlc_save
uv run export_community.py --auto
```

The browser window will appear on your local machine's display.

Option 5: VNC Server

Set up a VNC server on RHEL 10 for remote GUI access:

```
# Install VNC server (one-time setup)
sudo dnf install tigervnc-server

# Start VNC session
vncserver :1

# Connect with VNC client, then run:
export DISPLAY=:1
uv run export_community.py --auto
```

Browser opens but shows "Page not found" or a blank page

The script previously navigated to `/t5/s/sso` which is not a valid URL on this site. This is fixed in the current version — the browser now opens at the homepage. If you see this with an older version, update the script.

Browser opens but I don't see a login form

This is expected. `learn.redhat.com` is a JavaScript SPA — there is no standalone login page. Click **Sign In** in the top-right corner of the homepage to start the Red Hat SSO login flow.

"Could not automatically locate the 'My community content' download button"

The script tried several known selectors and text patterns but could not find the download link. The terminal will log all links/buttons found on the page to help diagnose the issue.

Manual workaround: The browser stays open. Navigate to:

```
https://learn.redhat.com/t5/user/myprofilepage/tab/personal-profile:advanced-profile
```

Scroll to the **Downloads** section and click **My community content**. Then press Enter in the terminal — the script will capture the downloaded file.

Images show as `[Image: <id>]` placeholders

- The image was not downloaded (auth failure or network error)
- Check `output/download_errors.log` for details
- Export fresh cookies and re-run: `uv run export_community.py --cookies cookies.txt`

`ModuleNotFoundError: No module named 'requests'`

Install dependencies:

```
uv sync
# or
pip install -r requirements.txt
```

`ModuleNotFoundError: No module named 'playwright'`

Install Playwright (required for `--auto` and `--fetch-json`):

```
uv pip install playwright
uv run playwright install chromium
```

`JSONDecodeError` on startup

The JSON file may be malformed. Validate it:

```
python3 -c "import json; json.load(open('my_community_content.json'))"
```

`JSON file not found: my_community_content.json`

You need to download your community content export first. Either:

- Run `uv run export_community.py --auto` to download it automatically, or
- Follow the Appendix steps to download it manually from the RHLC website

Syntax highlighting not working offline

The `output/assets/highlight.min.js` file may not have downloaded. Delete it and re-run (without `--skip-assets`):

```
rm -rf output/assets/
uv run export_community.py --skip-images --skip-attachments --no-auth
```

Cookies expired mid-run

If you see auth errors (401/403) in `download_errors.log` after a run that previously worked, your cookies have expired. Export fresh cookies from your browser and re-run — already-downloaded files will be skipped.

Post Filename Format

Each post is saved as `output/posts/<Section>-<Subject>.html`.

- Section and subject are slugified (spaces to underscores, special chars removed)
- Duplicate titles get `_2`, `_3` suffixes
- Maximum filename length: ~100 characters

Example: A post titled *"How to use Ansible"* in the *"Automation"* forum becomes `output/posts/Automation-How_to_use_Ansible.html`.

Supported Khoros Tags

The following Khoros-specific HTML tags are transformed to standard HTML:

Khoros Tag	Rendered As
`<li-image id="...">`	`<img src="../images/...">`
`<li-code lang="python">`	`<pre><code class="language-python">`
`<li-user uid="123">`	`<span class="user-mention">@user_123</span>`
`<li-emoji id="lia_rocket">`	rocket emoji

Appendix: How to Export Your Community Content from RHLC

> **Note:** RHLC (Red Hat Learning Community at learn.redhat.com) is scheduled for decommissioning in Q2. Follow these steps to download your content as a JSON file before the site is shut down.

Step 1 — Log in to RHLC

Navigate to https://learn.redhat.com/ and log in with your Red Hat account.

Step 2 — Open My Settings

Click your **profile picture** in the upper-right corner of the page and select **My Settings** from the dropdown menu.

Step 3 — Open Advanced Profile

In the settings sub-menu, click **Advanced Profile**. Scroll to the bottom of the page.

Step 4 — Download My Community Content

In the **Downloads** section at the bottom of the Advanced Profile page, click **My community content**. The site will generate and download a JSON file containing all of your posts, replies, and associated image URLs.

> **Note:** It is unclear whether the JSON export includes content from every space you have participated in. The file can be inspected and manipulated after download if needed.

Step 5 — Save the JSON file

Save the downloaded file as `my_community_content.json` in the same directory as `export_community.py` (i.e., this repository root).

The file is listed in `.gitignore` and will **not** be committed to the repository. A sample file (`my_community_content.sample.json`) is provided in the repository to demonstrate the expected format.

Step 6 — Run the exporter

Once you have `my_community_content.json`, follow the Quick Start instructions at the top of this guide to generate your offline HTML archive:

```
# Fully automatic (downloads JSON + runs export in one step):
uv run export_community.py --auto

# With browser cookie authentication (if JSON already downloaded):
uv run export_community.py --cookies cookies.txt

# Or with Playwright browser login (if JSON already downloaded):
uv run export_community.py --save-cookies
```

> **Shortcut:** If you have Playwright installed, you can skip Steps 2–5 entirely and just run `uv run export_community.py --auto`. The script will open a browser, wait for you to log in, then navigate to your Advanced Profile and download the JSON automatically before running the full export.